

MIDTOWN MOTORS USED CARS APPLICATION

CS 6400 Phase 1 Report – Team 15

Ugochukwu Onuoha (uonuoha3)
Mattis Voss (mvoss32)
Monroe Stephenson (mstephenson31)
Andre Dingiswayo (adingiswayo3)
Jose Antonio Rodriguez Albala (jalbala3)

Table of Contents

Table of Contents	1
1 Data Types	2
2 Business Logic Constraints	5
2.1 Customer Constraints	5
2.2 Staff Constraints.....	5
2.3 Vehicle Constraints	5
2.4 Sale Price Constraints	6
2.5 Parts Order Constraints	6
2.6 Vendor Constraints	7
2.7 Acquisition Specialist Constraints	7
2.8 Sales Agent Constraints	7
2.9 Operating Manager Constraints	8
2.10 Owner Constraints.....	8
2.11 Reports Constraints	8
2.12 Search Constraints (Public & Staff)	9
3 Task Decomposition with Abstract Code.....	11
3.1 Search Vehicles	11
3.2 Authenticate User.....	15
3.3 View Vehicle Details.....	15
3.4 Modify Part Status	18
3.5 Confirm Sale of Vehicle.....	20
3.6 Lookup/Add Customer.....	21
3.7 Add Vehicle	23
3.8 Lookup/Add Vendor.....	24
3.9 Add Part to Order	26
3.10 Generate Seller History Report.....	27
3.11 Generate Avg Time in Inventory Report	29
3.12 Generate Price Per Condition Report	30
3.13 Generate Parts Statistics Report	31
3.14 Generate Monthly Sales/Drilldown Report	32

1 Data Types

Entity: STAFF_USER

Attribute	Data type	Nullable
Password_plaintext	string	Not null
First_name	string	Not null
Last_name	string	Not null
Username	string	Not null

Entity: CUSTOMER (supertype)

Attribute	Data type	Nullable
Customer_identifier	string	Not null
Street	string	Not null
City	string	Not null
State	string	Not null
Postal_code	string	Not null
Phone	string	Not null
Email	string	Null

Entity: INDIVIDUAL_CUSTOMER (subtype)

Attribute	Data type	Nullable
First_name	string	Not null
Last_name	string	Not null
SSN	string	Not null

Entity: BUSINESS_CUSTOMER (subtype)

Attribute	Data type	Nullable
Business_name	string	Not null
Primary_contact_first_name	string	Not null
Primary_contact_last_name	string	Not null
Primary_contact_title	string	Not null
Tax_ID	string	Not null

Entity: VEHICLE

Attribute	Data type	Nullable
Vin	String	Not null
Model_name	String	Not null
Model_year	Integer	Not null

Fuel_type	List<string>	Not null
Drivetrain	List<string>	Not null
Horsepower	Integer	Not null
Notes	string	Null
Condition	List<string>	Not null
Sale_price	Float	Not null

Entity: VEHICLE_TYPE

Attribute	Data type	Nullable
Type_name	String	Not null

Entity: MANUFACTURER

Attribute	Data type	Nullable
Manufacturer_name	String	Not null

Entity: COLOR

Attribute	Data type	Nullable
Color_name	String	Not null

Entity: PURCHASE

Attribute	Data type	Nullable
Vin	String	Not null
Purchase_date	Date	Not null
Purchase_price	Float	Not null

Entity: SALE

Attribute	Data type	Nullable
Vin	String	Not null
Sale_date	Date	Not null

Entity: VENDOR

Attribute	Data type	Nullable
Vendor_name	String	Not null
Street	String	Not null
City	String	Not null
State	String	Not null

Postal_code	String	Not null
Phone	String	Not null

Entity: PARTS_ORDER

Attribute	Data type	Nullable
Parts_order_number	String	Not null
Order_ordinal	Integer	Not null

Entity: PARTS_ORDER_LINE

Attribute	Data type	Nullable
Parts_order_number	String	Not null
Vendor_part_number	String	Not null
Description	String	Not null
Unit_price	Float	Not null
Quantity	Integer	Not null
Status	List<string>	Not null

2 Business Logic Constraints

2.1 Customer Constraints

A customer may be an individual or a business, not both simultaneously.

All customers must have:

- Address (street, city, state, postal code)
- Phone number
- Optional email address

For individual customers:

- SSN must be unique.
- First and last name are required.

For business customers:

- Tax ID must be unique.
- Business name is required.
- Primary contact's First name, Last name, and Title are required.

Also:

- A customer may serve as both seller and buyer across different transactions.
- The same customer record must be reused for all activity.
- Customer lookup is only permitted in the context of a purchase or sale transaction; it is not independently accessible as a standalone feature.
- Customers may be looked up by SSN (individual) or Tax ID (business) only.
- If a customer is not found by lookup, the system must present the option to add a new customer; the appropriate fields are collected based on customer type.

2.2 Staff Constraints

Each staff user must belong to at least one role (Total ISA). The specialization is modeled as Overlapping to allow the Owner to participate in all role-specific relationships. In practice the four roles are:

- Acquisition Specialist
- Sales Agent
- Operating Manager
- Owner

The Owner can perform all activities allowed to any other role.

2.3 Vehicle Constraints

- Each vehicle is purchased by Midtown Motors exactly once and sold at most once.
- VIN is alphanumeric and must be unique.
- Model year must be exactly 4 digits and must not exceed the current calendar year plus one.
- Model name is free-form text entered by the user.

- Notes is an optional free-form text field.
- Horsepower must be a positive whole number (integer).
- Condition must be exactly one of the following fixed values: Excellent, Very Good, Good, Fair, Rough
- Fuel type must be exactly one of the following fixed values: Gas, Diesel, Natural Gas, Hybrid, Plugin Hybrid, Battery, Fuel Cell
- Drivetrain must be exactly one of the following fixed values: 4WD, AWD, FWD, RWD
- Vehicle type is selected from a DBA-maintained list; new types may be added by the DBA.
- Manufacturer is selected from a DBA-maintained list; the list may be updated by the DBA.
- Color is selected from a fixed list that is not expected to change; a vehicle must have at least one color and may have multiple colors.
- Purchase price is manually entered by the acquisition specialist; it is not calculated.
- The acquisition specialist manually assigns the vehicle's condition at time of purchase.
- Purchase date must be recorded for every vehicle acquisition (used for time-in-inventory and seller history calculations).

2.4 Sale Price Constraints

Sale price is a derived value and must not be manually entered or modified:

$$\text{Sale Price} = (1.25 \times \text{purchase_price}) + (1.10 \times \text{total_parts_cost})$$

where $\text{total_parts_cost} = \text{sum of } (\text{unit_price} \times \text{quantity})$ for all parts lines on all parts orders for that vehicle.

Midtown Motors does not negotiate price; the calculated sale price is final.

2.5 Parts Order Constraints

- Parts order numbers are generated by the system, not entered by the user, using the format: VIN concatenated with the ordinal of the order for that vehicle, padded to exactly three digits.
 - Example:
 - first order for VIN ABCDEFGHIJ1234567 → ABCDEFGHIJ1234567001
 - second order → ABCDEFGHIJ1234567002
- A parts order must be linked to exactly one vendor.
- A parts order must contain at least one part line item.
- It is permissible for a vehicle to have multiple parts orders from the same vendor.
- Each part line item within an order tracks: a description, a vendor part number (which may be alphanumeric), a unit price, a quantity, and a status.
- All units of the same part kind within a single parts order share one status value; they are always ordered, received, and installed together.
- Part prices are stored per order line at the time of ordering. No static catalog of part numbers and prices is maintained, because part prices may change over time.

- A vehicle with any parts order line not yet at Installed status is considered to have parts pending and must not appear in public search results or be eligible for sale.

2.6 Vendor Constraints

- Vendor names are unique and serve as the natural identifier.
- Vendor lookup and add must be accessible from the parts order form only, parallel to how customer lookup works during purchase/sale.
- If no vendor is found by name, the user may add a new vendor with: Name, street address, city, state, postal code, and phone number.

2.7 Acquisition Specialist Constraints

- Ability to add Vehicles:
 - Must search for or add a customer before recording a purchase.
 - Must provide all required vehicle attributes and purchase data.
 - Must see full vehicle details including purchase price and parts total.
 - The search screen must display both the count of vehicles with parts pending and the count of vehicles available for sale.
- Parts Management:
 - Can create new parts orders for a vehicle.
 - Each parts order must be linked to exactly one vendor.
 - May add multiple part kinds to a single order.
- Part Status Rules:
 - Valid status transitions are:
 - Ordered -> Received -> Installed
 - Ordered -> Installed (direct jump is permitted)
 - Status cannot revert to a previous value.
 - Once Installed, status cannot be changed.
 - Each part line item's status is tracked individually within an order.

2.8 Sales Agent Constraints

- Search:
 - Can only search for vehicles that:
 - Have no pending (uninstalled) parts, and
 - Are not yet sold.
- Sales:
 - To perform a sale, must search for or add a customer.
 - Sale date must be recorded.
 - Sale price is calculated by the system and cannot be overridden.
 - If a buyer acquires multiple vehicles at the same time, each vehicle is handled as a separate, independent sale transaction; there is no multi-vehicle sale.

2.9 Operating Manager Constraints

View Permissions:

- Can view all data, including:
 - Seller contact info (everything except SSN/Tax ID)
 - Buyer contact info (everything except SSN/Tax ID)
 - Name (first and last) of the acquisition specialist who handled the purchase
 - Name (first and last) of the sales agent who executed the sale (if sold)
 - Purchase info, explicitly including: purchase price and purchase date
 - Sale info, explicitly including: sale date
 - Parts info
 - The search screen must display both the count of vehicles with parts pending and the count of vehicles available for sale.

Search:

- Can filter vehicles by:
 - Sold — returns only sold vehicles
 - Unsold — returns all unsold vehicles regardless of parts status
 - All — returns all vehicles; unsold vehicles are included regardless of parts status
- When filtering by Unsold or All, parts pending status does not exclude a vehicle.

2.10 Owner Constraints

Permissions:

- Can perform every action available to:
 - Acquisition specialists
 - Sales agents
 - Operating managers
- The vehicle detail page shows all information as it does for operating managers, and the Owner may additionally sell a vehicle or add parts to it.

2.11 Reports Constraints

Reports are accessible only to the Owner and Operating Managers.

The following reports must be supported:

- Seller History Report:
 - Displays per seller: seller name (individual: "first last"; business: business name shown as a single column), total vehicles purchased from them, average purchase price, average total quantity of all parts per vehicle, and average total parts cost per vehicle.
 - Sorted by total vehicles descending, then average purchase price ascending.
 - Any row where average total parts quantity ≥ 5 OR average parts cost $\geq \$500$ must be highlighted with a red background.

- Average Time in Inventory Report:
 - Displays average days in inventory per vehicle type, for sold vehicles only.
 - Days in inventory = Sale Date - Purchase Date + 1 (minimum 1 day).
 - Vehicle types with no sales history display "N/A".
 - Sorted by vehicle type ascending.

- Price Per Condition Report:
 - Matrix report: rows are conditions (Excellent, Very Good, Good, Fair, Rough) in descending order; columns are vehicle types in alphabetical order.
 - Cells show the average purchase price paid by Midtown Motors.
 - If no vehicles of that type/condition have been purchased, display "\$0".

- Parts Statistics Report:
 - Displays per vendor: vendor name, total quantity of all parts supplied, total dollar amount spent on parts from that vendor.
 - Sorted by vendor name ascending.

- Monthly Sales Report:
 - Summary page: lists by year and month — total vehicles sold, gross sales income, and net income (sale price minus purchase price minus total parts cost).
 - Sorted by year and month descending (most recent first).
 - Months with no sales may be excluded.
 - Each year/month row links to a drilldown report showing, for that month:
 - Sales agent first and last name, number of vehicles sold, total sales amount.
 - Sorted by number of vehicles sold descending, then total sales amount descending.

2.12 Search Constraints (Public & Staff)

Public Search:

- The landing page must prominently display the total count of vehicles currently available for sale (no pending parts, not yet sold).
- Search criteria include:
 - Vehicle type
 - Manufacturer
 - Year
 - Fuel type
 - Drivetrain
 - Color
 - Keyword (searches manufacturer, model, year, and notes fields)

- Only single selections allowed for dropdowns (no multi-select).
- No selection in a dropdown means all values for that field are acceptable (no filter applied).
- Results must match ALL entered search criteria simultaneously (AND logic across all fields).
- Keyword search must be a case-insensitive substring match against all four fields.
- Results must only include vehicles with all parts installed and not yet sold.
- If no vehicles match, display: "Sorry, it looks like we don't have that in stock!"
- If vehicles match, the result set must include all colors for a vehicle in a single row.
- Results are sorted by VIN ascending; the user cannot override the sort order.

Staff Search:

- All staff can additionally search by VIN (case-insensitive).
- Acquisition specialists can see all unsold vehicles, including those with parts pending.
- Operating managers and the Owner can filter by Sold, Unsold, or All (see above).

3 Task Decomposition with Abstract Code

3.1 Search Vehicles

Task Decomposition

Lock Types:

1. The calculation of the number of vehicles available for sale, and the vehicles with parts pending, will require read-only locks on [VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#) and [SALE](#).
2. Common vehicle attributes search (Vehicle type, Manufacturer, Year, Fuel type, Drivetrain layout, Color, and VIN attributes) will require read-only locks on [VEHICLE](#), [PARTS_ORDER](#), and [PARTS_ORDER_LINE](#), [TYPE](#), and [MANUFACTURER](#), and [SALE](#).
3. Privileged searches from Operating Managers will require the read-only locks on [VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), [TYPE](#), [MANUFACTURER](#), and [SALE](#).
4. Keyword search (Manufacturer, Model, Year, and Notes attributes) will require read-only locks on [VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), [MANUFACTURER](#), and [SALE](#).

Number of Locks: calculation for vehicles available requires 4 locks, common vehicle attributes search requires 6 read locks, privileged search requires 6 locks, and keyword search requires 5 read locks.

Enabling Conditions: none for regular users and successful login (correct username and password credentials) for privileged users.

Frequency:

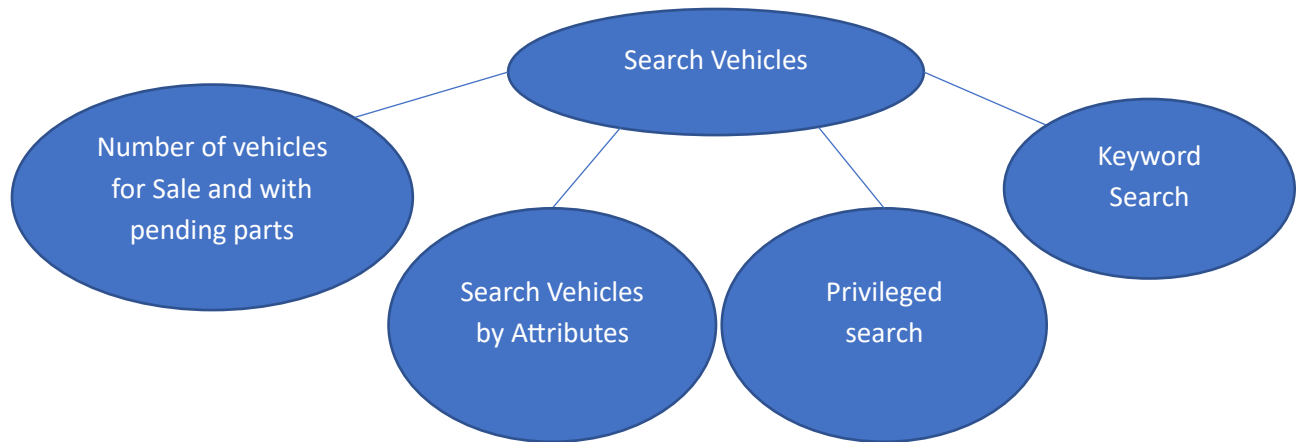
- Inventory calculation: high (available number of cars calculated every time users land on the search page).
- Common vehicle attributes search: medium-high (users are expected to perform quite few queries while looking for the right car).
- Keyword searches: low frequency high (residual use when other searches do not work), but intensive use of resources.

Consistency (ACID): not critical. It is not critical if we miss one vehicle in the search results, it will eventually appear in subsequent searches.

Subtasks:

1. Number of vehicles for sale and with pending parts: separate task because it requires [VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), and [SALE](#) locks and is of high frequency.
2. Search vehicles by Attributes: separate task because it requires [VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), [TYPE](#), [MANUFACTURER](#), and [SALE](#) locks and is of medium-high frequency.
3. Privileged search: separate task because of different enabling conditions, as compared to regular search vehicles by attributes subtask.
4. Keyword search: separate task because of its low frequency and different locks required ([VEHICLE](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), [MANUFACTURER](#), and [SALE](#)). This task will also make intensive use of computing resources.

As a result, a mother task to control all these subtasks is necessary.



Abstract Code

- ❖ The **Vehicle Search/Result Form** will be the front page of the application. It has public access for regular users to perform the search for a car based on certain parameters.
- ❖ Privileged users will be able to log in to access restricted information that is not shown to regular users by authenticated with a valid username and password and clicking on the Login button.
- ❖ Run the **Search Vehicles** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - Find all **VEHICLE** records that have not been sold via the SOLD_VEHICLE relationship towards **SALE**, and have no pending parts via the FOR_VEHICLE relationship towards **PARTS_ORDER** and the EXISTS_IN relationship towards **PARTS_ORDER_LINE**:
 - For regular and privileged users calculate the number of vehicles available for sale by counting all **VEHICLE.VIN** after excluding the number of vehicles that have **SALE.Sale_date**, and all unique **PARTS_ORDER.Parts_Order_Number** (Parts_Order_Number = VIN) where there is still at least one **PARTS_ORDER_LINE.Status** in “ordered” or “received” status (or different to installed).
 - For privileged users acquisition specialist, operating managers, and the owner, the number of vehicles with missing parts will be calculated as the count of unique **PARTS_ORDER.Parts_Order_Number** (VIN) where there is still at least one **PARTS_ORDER_LINE.Status** in “ordered” or “received” status (or different to installed).
 - The number of cars for sale and the number of cars with missing parts will be shown on the Search Vehicles page depending on the user that accesses it.
 - Find all **VEHICLE** records associated with a **TYPE** of car via IS_OF_TYPE relationship, or a **MANUFACTURER** via MADE_BY relationship, or **VEHICLE** attributes (Year, Fuel_type, Drive_train, or Color). For filtering purposes, **SALE** data will be obtained via SOLD_VEHICLE relationship, as well as **VEHICLE** records that have pending parts via the FOR_VEHICLE relationship towards **PARTS_ORDER** and the EXISTS_IN relationship towards **PARTS_ORDER_LINE**.
 - Any users (public/privileged) can search for following common vehicle attributes: **VEHICLE.MANUFACTURER.Manufacturer_name**, **VEHICLE.TYPE.Type_name**, **VEHICLE.Year**, **VEHICLE.Fuel_type**,

VEHICLE.Drive_train, VEHICLE.Color. Only one parameter at a time can be used for the search. All these attributes will be shown on the Vehicle Search page when the user lands on it. To ease the search, all these attributes will provide a drilldown functionality only showing unique values found among the recorded attributes.

- When the search criteria are fixed, the user will click on **Search** button.
 - This search will only show *vehicles on sale*: all VEHICLE.VIN excluding all PARTS_ORDER.Parts_Order_Number (Parts_Order_Number = VIN) where there is still at least one PARTS_ORDER_LINE.Status in “ordered” or “received” status (or different to installed) and all VEHICLE.VIN that have associated a SALE.Sale_date.
 - The output of a search will be a list of vehicles with no parts pending and following attributes: VEHICLE.MANUFACTURER.Manufacturer_name, VEHICLE.TYPE.Type_name, VEHICLE.VIN, VEHICLE.Model, VEHICLE.Year, VEHICLE.Fuel_type, VEHICLE.Drive_train, VEHICLE.Color, VEHICLE.Horse_power, and VEHICLE.Sale_price. When VEHICLE.Color stores several colors, only one row with all colors listed will be returned. These results are sorted by VIN in ascending order. By the side of each car, the user can click on **Vehicle Details** and the **Vehicle Details Form** (detail page) will pop up.
 - When no vehicles match the searching criteria, the error message received is: “Sorry, it looks like we don’t have that in stock will be shown!”
- For the privileged search we will also need to find all VEHICLE records associated with a TYPE of car via IS_OF_TYPE relationship, or a MANUFACTURER via MADE_BY relationship, or VEHICLE attributes (Year, Fuel_type, Drive_train, or Color). For filtering purposes, SALE data will be obtained via SOLD_VEHICLE relationship, as well as VEHICLE records that have pending parts via the FOR_VEHICLE relationship towards PARTS_ORDER and the EXISTS_IN relationship towards PARTS_ORDER_LINE.
- All privileged users can perform a search based on common vehicle attributes, as regular users can (VEHICLE.MANUFACTURER.Manufacturer_name, VEHICLE.TYPE.Type_name, VEHICLE.Year, VEHICLE.Fuel_type, VEHICLE.Drive_train, VEHICLE.Color):
 - Sales agents and: gets only *vehicles on sale*.
 - Acquisitions specialists: get vehicles on sale and vehicles with pending parts (all VEHICLE.VIN except those that have associated a SALE.Sale_date).
 - Owner: full access.
 - Moreover, all privileged can perform a search in the database matching a particular VIN. A designated field for this purpose will be shown in the Search page. When the privileged user introduces the VIN (case sensitive), all VEHICLE.VIN will be searched for a match.
 - Operating managers can filter by sold vehicles, unsold vehicles, or all vehicles:
 - Sold vehicles: all VEHICLE.VIN that have associated a valid SALE.Sale_date.

- Unsold vehicles: all [VEHICLE.VIN](#) that do not have associated a valid [SALE.Sale_date](#).
 - All vehicles: all [VEHICLE.VIN](#) in the database.
- When the search criteria are fixed, the user will click on **Search** button.
- In all cases, the search request will provide [VEHICLE.MANUFACTURER.Manufacturer_name](#), [VEHICLE.TYPE.Type_name](#), [VEHICLE.VIN](#), [VEHICLE.Model](#), [VEHICLE.Year](#), [VEHICLE.Fuel_type](#), [VEHICLE.Drive_train](#), [VEHICLE.Color](#), [VEHICLE.Horse_power](#), and [VEHICLE.Sale_price](#). When [VEHICLE.Color](#) stores several colors, only one row with all colors listed will be returned. These results are sorted by VIN in ascending order. By the side of each car, the user can click on **Vehicle Details** and the **Vehicle Details Form** (detail page) will pop up.
- When no vehicles match the searching criteria, the error message received is: “Sorry, it looks like we don’t have that in stock will be shown!”
- Find all [VEHICLE](#) records that match a keyword entered by the user across [MANUFACTURER](#) via MADE_BY relationship, or [VEHICLE](#) attributes (Year, Model, or Notes). For filtering purposes, [SALE](#) data will be obtained via SOLD_VEHICLE relationship, as well as [VEHICLE](#) records that have pending parts via the FOR_VEHICLE relationship towards [PARTS_ORDER](#) and the EXISTS_IN relationship towards [PARTS_ORDER_LINE](#).
 - The user will provide a keyword (case sensitive) in the designated field for that purpose in the Search Page. When the search criteria are fixed, the user will click on **Search** button.
 - The keyword is then compared with all words stored in [VEHICLE.MANUFACTURER.Manufacturer_name](#), [VEHICLE.Model](#), [VEHICLE.Year](#), and [VEHICLE.Notes](#) parameters that the user could access to:
 - Public users and sales agents we will just get the *vehicles on sale*.
 - Acquisition managers: vehicles on sale and vehicles with missing parts (unsold vehicles).
 - Operating managers: sold, unsold and all vehicles.
 - Owner: full access.
 - In all cases, the search request will provide [VEHICLE.MANUFACTURER.Manufacturer_name](#), [VEHICLE.TYPE.Type_name](#), [VEHICLE.VIN](#), [VEHICLE.Model](#), [VEHICLE.Year](#), [VEHICLE.Fuel_type](#), [VEHICLE.Drive_train](#), [VEHICLE.Color](#), [VEHICLE.Horse_power](#), and [VEHICLE.Sale_price](#) for the vehicles that have a match in any of the attributes with the keyword provided by the user. When [VEHICLE.Color](#) stores several colors, only one row with all colors listed will be returned. These results are sorted by VIN in ascending order. By the side of each car, the user can click on **Vehicle Details** and the **Vehicle Details Form** (detail page) will pop up.
 - When no vehicles match the searching criteria, the error message received is: “Sorry, it looks like we don’t have that in stock will be shown!”

3.2 Authenticate User

Task Decomposition

Lock Types: read-only lock types on [STAFF](#).

Number of Locks: single.

Enabling Conditions: none.

Frequency: log in of users is expected to be of low frequency (once a day per user).

Consistency (ACID): critical, privileged users should not be allowed to authenticate using outdated passwords.

Subtasks: no mother task/subtask decomposition is needed.



Abstract Code

- ❖ Privileged users can authenticate to access specific features associated with each role by filling username and password in the designated fields shown in the [Vehicle Search/Result Form](#) and then pressing the Login button.
- ❖ Run the **Authenticate User** task as follows for each login attempt:
 - Obtain username ('\$Username') and password ('\$Password') introduced by the user when the Login button is pressed.
 - If [STAFF](#).User_name = '\$Username' and [STAFF](#).Password = '\$Password' then allow access for the privileged user:
 - Record session information and user role ([ACQUISITION_SPECIALIST](#)/[SALES_AGENT](#)/[OPERATING_MANAGER](#)/[OWNER](#)) and refresh [Vehicle Search/Result Form](#) allowing access to appropriate functionalities.
 - If [STAFF](#).User_name != '\$Username' or [STAFF](#).User_name = '\$Username' and [STAFF](#).Password != '\$Password':
 - Refresh [Vehicle Search/Result Form](#) and provide generic login error message: "Incorrect username or password".

3.3 View Vehicle Details

Task Decomposition

Lock Types: All locks are read-only, but affect three database regions. Basic vehicle characteristics, [PURCHASE](#) and [PARTS_ORDER](#) information, and [SALE](#) and [CUSTOMER](#) information.

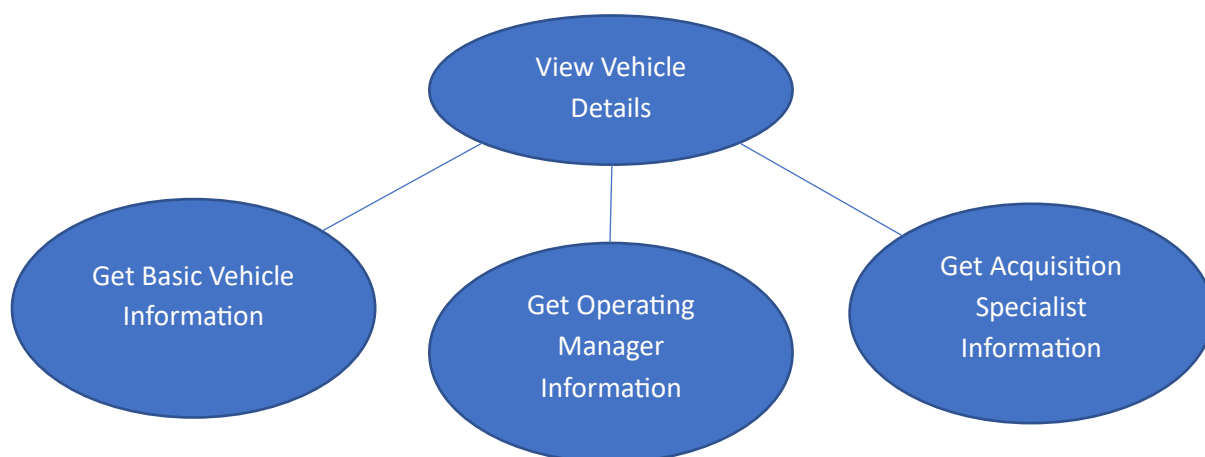
Number of Locks: Minimum 4 read locks for basic display ([VEHICLE](#), [VEHICLE_TYPE](#), [MANUFACTURER](#), [COLOR](#)). [ACQUISITION_SPECIALIST](#) view adds 6 more ([PURCHASE](#), [CUSTOMER](#), [STAFF](#), [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), [VENDOR](#)) for 10 total. Management view adds 3 more ([SALE](#), [CUSTOMER](#), [STAFF](#)) for 13 total. [HAS_COLOR](#) is many-to-many so requires one additional read lock on the junction table. Maximum locks: 14 read locks for Owner viewing sold vehicle with full details.

Enabling Conditions: First lookup always occurs; the others are enabled based on the category of user. Basic vehicle information requires only that VIN exists, accessible to all users including public. Acquisition specialist information requires user role of [ACQUISITION_SPECIALIST](#) or [OWNER](#) and vehicle must be unsold. Operating manager information requires user role of [MANAGER](#) or [OWNER](#).

Frequency: Basic vehicle information is high frequency and accessed by all user types (public, sales agents, acquisition specialists, managers, owner). Acquisition specialist information is medium frequency—accessed only by acquisition staff managing inventory and parts. Operating manager information is probably lower frequency, accessed only by management for review. Indexing should prioritize high-frequency reads ([VIN](#), [TYPE](#), [MANUFACTURER](#), [COLOR](#)).

Consistency (ACID): Not critical. Read-only operation with acceptable stale data if concurrent updates occur.

Subtasks: 3 subtasks; mother task needed to coordinate conditional execution. **View Vehicle Details** receives VIN and user role, always executes **Get Basic Vehicle Information**, conditionally executes **Get Acquisition Specialist Information** (if Acquisition Specialist or Owner role and vehicle unsold), conditionally executes **Get Operating Manager Information** (if Manager or Owner role).



Abstract Code

- ❖ User navigated to the **Vehicle Details** form associated with the **View Vehicle Details** task by selecting a search result on the **Vehicle Search** form, and clicking button **Vehicle Details**. The [VEHICLE.VIN](#) attribute gets passed in to the **View Vehicle Details** Mother task.
- ❖ The **Mother** task calculates the `Sale_price`:

- Find the current **VEHICLE** using **VEHICLE.VIN**
- Find the **PARTS_ORDER** entities associated with the current **VEHICLE**.
- For each **PARTS_ORDER**:
 - Find all **PARTS_ORDER_LINES**.
- Calculate the Total_parts_cost using the Unit_price and Quantity attributes of each **PARTS_ORDER_LINE**.
- Find the **PURCHASE** associated with the **VEHICLE** via PURCHASED_VEHICLE to get the Purchase_price.
- Calculate the Sale_price. The formula is:
 - $\text{Sale_price} = 1.25 \times \text{Purchase_price} + 1.10 * \text{Total_parts_cost}$
- ❖ The **Mother** task runs the **Get Basic Vehicle Information** task, passing in the **VEHICLE** and Sale_price:
 - Query for information about the **Vehicle**:
 - Display the **VEHICLE** single-value attributes of VIN, Model, Year, Fuel_type, Drivetrain, Horsepower, Notes
 - Find the **COLOR(s)** of the **VEHICLE** via HAS_COLOR and display all Color_name attributes in a single result row list.
 - Find the **TYPE** of the **VEHICLE** via IS_OF_TYPE and display the Type_name attribute.
 - Find the **MANUFACTURER** of the **VEHICLE** via MADE_BY and display Manufacturer_name attribute.
 - Display Sale Price.
 - If the user is not logged in, we are done and can wait for a click on **Back to Search** button.
- ❖ If the user is logged in as a **SALES_AGENT**:
 - If the **VEHICLE** is unsold and all **PARTS_ORDER_LINE** records have status = 'installed', display the **Sell Vehicle** button.
 - On click **Sell Vehicle button**, go to the **Lookup/Add Customer** form, passing **VEHICLE.VIN**.
- ❖ If the user is logged in as an **ACQUISITION_SPECIALIST**, the mother task runs the **Get Acquisition Specialist Information** task, passing the **VEHICLE** and information on **PARTS_ORDER(s)** **PARTS_ORDER_LINES(s)** and Total_parts_cost:
 - Find the **PURCHASE** associated with the current **VEHICLE** via PURCHASED_VEHICLE, and display Purchase_price and Purchase_date.
 - For each **PARTS_ORDER_LINE**:
 - Display the Vendor_part_number, Descriptions, Unit_price, Quantity, and Status attributes, as well as the **PART_ORDER** it came from.
 - Find the **VENDOR** associated with each **PARTS_ORDER** via FROM_VENDOR and display Vendor_name.
 - If status is not 'installed', display the **Update Status** button for that **PARTS_ORDER_LINE** row.
 - Display **Add Parts Order** button.
 - On click **Update Status button**: Go to the **Modify Part Status** form, passing Parts_order_number and Vendor_part_number.
 - On click **Add Parts Order** button: Go to the **Add Parts Order** form, passing **VEHICLE.VIN**.

- ❖ If the user is logged in as an **OPERATING_MANAGER**, run the **Get Operating Manager Information** task, passing in the **VEHICLE**:
 - Find the **CUSTOMER** from whom the **VEHICLE** was bought via FROM_SELLER.
 - If **CUSTOMER** is an **INDIVIDUAL_CUSTOMER**, display First_name and Last_name.
 - If **CUSTOMER** is a **BUSINESS_CUSTOMER**, display Business_name.
 - Display **CUSTOMER** Street, City, State, Postal_code, Phone, Email. Do not display Ssn or Tax_id.
 - If a **SALE** record exists for this **VEHICLE** via SOLD_VEHICLE:
 - Display **SALE**.Sale_date and the derived Sale_price.
 - Find the **SALES_AGENT** who executed the **SALE** via EXECUTED_BY and display First_name and Last_name.
 - Find the **CUSTOMER** to whom the **VEHICLE** was sold via TO_BUYER.
 - If the **CUSTOMER** is an **INDIVIDUAL_CUSTOMER**, display First_name and Last_name.
 - If the **CUSTOMER** is a **BUSINESS_CUSTOMER**, display Business_name.
 - Display buyer Street, City, State, Postal_code, Phone, Email.
 - Do not display Ssn or Tax_id.
 - Find and display the Acquisition Specialist associated with the Vehicle via HANDLED_BY.
 - No action buttons are displayed. When ready, **OPERATING_MANAGER** user clicks the Back to Search button to return to the Vehicle Search form.
- ❖ If the user is logged in as an **OWNER**:
 - Run the **Get Acquisition Specialist Information** task passing in **VEHICLE** and information on **PARTS_ORDER(s)** **PARTS_ORDER_LINE(s)** and Total_parts_cost,
 - Run the **Get Operating Manager Information** task passing in the **VEHICLE**.
 - If the **VEHICLE** is unsold and all **PARTS_ORDER_LINE** records have status = 'installed', display the **Sell Vehicle** button.
 - As per tasks above:
 - On click **Sell Vehicle button**, go to the **Lookup/Add Customer** form, passing **VEHICLE.VIN**.
 - On click **Update Status button**: Go to the **Modify Part Status** form, passing parts_order_number and vendor_part_number.
 - On click **Add Parts Order** button: Go to the **Add Parts Order** form, passing **VEHICLE.VIN**.
- ❖ Finally for all users:
 - On click **Back to Search** button: Return to the **Vehicle Search** form (all user categories).

3.4 Modify Part Status

Task Decomposition

Lock Types: Read (for current status) followed by Write (for updated status) locks are required on PARTS_ORDER_LINE (the entity that describes a particular part in a **PARTS_ORDER**).

Number of Locks: There is a single read and a single write lock in the same entity (**PARTS_ORDER_LINE**), so the schema construct count is very low.

Enabling Conditions: User role must be [ACQUISITION_SPECIALIST](#) or [OWNER](#). Part status must not be 'installed' (installed parts cannot be updated per spec). VIN context must be valid (user arrived from [View Vehicle Details](#)). All conditions apply uniformly to the entire update operation.

Frequency: Medium frequency, the task gets triggered whenever Acquisition Specialists receive orders or install parts.

Consistency (ACID): Read and write must occur within a single ACID transaction to make sure that the transition remains valid (forward-only, i.e. ordered-to-received, ordered-to-installed, received-to-installed).

Subtasks: No decomposition, no mother task needed. Atomic update on a single part line with validation logic. Read (get current status), validate (check forward-only transition), and write (update status) must execute as a single transaction.



Abstract Code

- ❖ User navigated to the Modify Part Status form associated with the Modify Part Status task by clicking the **Update Status** button on the [Vehicle Details](#) form. The Parts_order_number and Vendor_part_number attributes are passed in from the [Vehicle Details](#) **View Vehicle Details** mother task.
- ❖ Run the **Modify Part Status** task as follows:
 - Find the current [PARTS_ORDER_LINE](#) using Parts_order_number and Vendor_part_number.
 - Display the current [PARTS_ORDER_LINE](#) attributes: Vendor_part_number, Description, Unit_price, quantity, status.
 - Find the [PARTS_ORDER](#) associated with this Parts_order_line via CONTAINS and display parts_order_number.
 - Find the [VENDOR](#) associated with the [PARTS_ORDER](#) via FROM_VENDOR and display vendor_name.
 - Display the *New Status* input field.
 - On click **Update Status** button:
 - Validate that new_status represents a valid forward transition from the current status. If not, display an error message and remain on the [Modify Part Status](#) form.
 - If valid, update Parts_order_line.status to new_status.
 - Return to the [Vehicle Details](#) form for current [Vehicle](#).
 - On click **Cancel** button:
 - Return to the [Vehicle Details](#) form for current [VEHICLE](#) without making any changes.

3.5 Confirm Sale of Vehicle

Task Decomposition

Lock Types: Mixture of read and write locks across different schema constructs around inventory and transaction history.

Number of Locks: 4 read locks needed. These are on **VEHICLE** (to confirm unsold), **PURCHASE** (get Purchase_price for calculation of Sale_price), **PART_ORDER_LINE** (to verify all parts installed and to calculate Sale_price), and **CUSTOMER**. Write lock needed on **SALE** (to insert the transaction). Relationships linking **SALE** to **CUSTOMER** and **STAFF** are 1:N, so no additional locks needed to implement these.

Enabling Conditions: Privileged user category must be **SALES_AGENT** or **OWNER**. Vehicle must be unsold (i.e. **SALE** does not exist for VIN). All parts must be 'installed'. **CUSTOMER** must be identified (Lookup/Add Customer task completed successfully with valid Customer_id).

Frequency: Low to medium frequency. Sales occur regularly but less frequently than viewing vehicles or updating parts status.

Consistency (ACID): The task must verify vehicle is unsold (availability check), verify parts are all installed, and record the transaction atomically. If another Sales Agent sells the vehicle between the availability check and the **SALE** insert, the transaction must fail. The verification reads and **SALE** write must occur within a single ACID transaction to prevent double-selling.

Subtasks: No subtasks identified, no mother task needed. must execute as a single atomic unit. Lookup/Add Customer is a separate predecessor task, not a subtask.



Abstract Code

- ❖ User navigated to the **Confirm Sale of Vehicle** form associated with the **Confirm Sale of Vehicle** task by clicking the **Sell Vehicle** button on the **Vehicle Details** form, proceeding through the **Lookup/Add Customer** task, and clicking the **Confirm** button on the **Lookup/Add Customer** form. **Vehicle.VIN** and **Sale_price** as well as **Customer_id** are passed in from the **Lookup/Add Customer** task.
- ❖ Run the **Confirm Sale of Vehicle** task as follows:
 - Find the current **VEHICLE** using VIN.
 - Verify that the Vehicle is unsold (no **SALE** record exists for this **VEHICLE** via **SOLD_VEHICLE**).
 - Verify that all **PARTS_ORDER_LINE** records associated with this **VEHICLE** have status = 'installed'.
 - If either condition is not met, display an error message and return to the **Vehicle Details** form.

- Find the **CUSTOMER** identified by the Lookup/Add Customer task, using the customer_id passed in from that task.
- Display VIN and the Sale_price.
- If the Customer is an **INDIVIDUAL_CUSTOMER**, display first_name and last_name.
- If the Customer is a **BUSINESS_CUSTOMER**, display business_name.
- Display the *Sale_date* input field.
- On click **Confirm Sale** button: Validate that Sale_date is a valid date. If not, display an error message and remain on the **Confirm Sale of Vehicle** form. If valid, insert a new **SALE** record with Sale_date, Sale_price, VIN, customer_id, and the current staff username via EXECUTED_BY. Return to the **Vehicle Details** form for the current **VEHICLE**.
- On click Cancel button: Return to the **Vehicle Details** form for the current **VEHICLE** without making any changes.

3.6 Lookup/Add Customer

Task Decomposition

Lock Types: Initial read lock on **CUSTOMER** to find correct entry via SSN or Tax_ID, depending on whether subtype is **INDIVIDUAL_CUSTOMER** or **BUSINESS_CUSTOMER**, respectively. If found, required information is retrieved and displayed. If adding new customer, write lock required on **CUSTOMER** to insert new record.

Number of Locks: This task only affects the **CUSTOMER** area of the database (**CUSTOMER** entity and one subtype), no other locks required. At most 1 read lock and 2 write locks (for **CUSTOMER** supertype and subtype).

Enabling Conditions: This task will only get called during an **Add Vehicle** or **Confirm Sale** transaction, which means that the user must be either SALES_AGENT, ACQUISITION_SPECIALIST or OWNER. The read lock will always be required, while the write lock will only be needed if a **CUSTOMER** entity with matching SSN or Tax_ID does not exist in the system.

Frequency: This task will be medium frequency. It will be lower than Vehicle Search or View Vehicle Details, but higher than the Report tasks, for example. The lookup will likely be somewhat higher frequency than the insert part of the task.

Consistency (ACID): The entire task must be atomic, otherwise a SALES_AGENT and an ACQUISITION_SPECIALIST (for example) could both pass the uniqueness check of SSN/Tax_ID, and try to insert the same CUSTOMER. If two staff members simultaneously try to add the same new customer, one must fail. The uniqueness check and insert must occur within a single transaction.

Subtasks: Based on the above discussion, this should remain a single task, zero subtasks, no mother task required. It is a single lookup-then-conditionally-insert operation on a single entity (with subtype). Although there is a difference in frequency between lookup and insert operations, the consistency requirement, enabling conditions, and small number of schemas all indicate that no decomposition should occur.

Abstract Code

- ❖ The **Lookup/Add Customer** task handles customer selection both when buying and selling a **VEHICLE**. An **ACQUISITION_SPECIALIST** or **OWNER** user interacts with this task via the **Add Vehicle** form. A **SALES_AGENT** or **OWNER** user interacts with this task via the **Confirm Sale** form. Both contexts present an interface allowing the user to search for a **CUSTOMER** by SSN or Tax_ID.
- ❖ Run the **Lookup/Add Customer** task as follows, passing in context (**Add Vehicle** or **Confirm Sale**):
 - Display search interface on the relevant form with radio buttons for search type (SSN or Tax_ID) and *SSN/Tax_id* input search field.
 - User selects search type (SSN or Tax_ID radio button) and enters value in search field.
 - On click Search button:
 - Check that radio button is selected and validate search field. If validation fails, show error message and remain on original form with search interface.
 - If searching by SSN, query **INDIVIDUAL_CUSTOMER** by SSN.
 - If searching by Tax_ID, query **BUSINESS_CUSTOMER** by tax_id.
 - If matching record found, display CUSTOMER details:
 - For **INDIVIDUAL_CUSTOMER**: Display First_name, Last_name, Street, City, State, Postal_code, Phone, and Email (optional).
 - For **BUSINESS_CUSTOMER**: Display Business_name, Primary_contact_first_name, Primary_contact_last_name, Primary_contact_title, Street, City, State, Postal_code, Phone, Email (optional).
 - On click:
 - **Confirm** button: Pass customer subtype (**INDIVIDUAL_CUSTOMER** or **BUSINESS_CUSTOMER**), and **CUSTOMER** key (SSN or Tax_id value) to the calling task (Add Vehicle or Confirm Sale of Vehicle, depending on context).
 - **Search Again** button: Return to original form with search interface.
 - If no record found, display **Add Customer** form with radio button to select **INDIVIDUAL_CUSTOMER** or **BUSINESS_CUSTOMER**. Radio button defaults to the search type originally selected. Display input fields:
 - Common fields for both types: Street, City, State, Postal_code, Phone, Email (optional).
 - If **INDIVIDUAL_CUSTOMER** selected: First_name, Last_name, SSN.
 - If **BUSINESS_CUSTOMER** selected: Business_name, Primary_contact_first_name, Primary_contact_last_name, Primary_contact_title, Tax_id.
 - On click:

- **Save** button: Validate all required fields are filled and uniqueness constraint is satisfied (SSN or Tax_id must not already exist in database). If validation fails, display error message and remain on **Add Customer** form. If valid, insert new **CUSTOMER** and subtype (**INDIVIDUAL_CUSTOMER** or **BUSINESS_CUSTOMER**) record. Pass **CUSTOMER** subtype and key (SSN or Tax_id value) to the calling task (**Add Vehicle** or **Confirm Sale**, depending on context).
 - **Cancel** button: Return to original context form with search interface.
- On click Cancel button:
- Clear inputs and remain on **Add Vehicle** or **Confirm Sale** form

3.7 Add Vehicle

Task Decomposition

Lock Types: Mixed locks across different database regions. Read locks on **MANUFACTURER** and **VEHICLE_TYPE** to validate dropdown selection. Read lock on **COLOR** for each selected color. Write lock on **VEHICLE** to insert new record. Write lock on **PURCHASE** to record the purchase transaction. HAS_COLOR relationship is M:N so needs write locks for each **VEHICLE-COLOR** pair being inserted.

Number of Locks: 3 read locks (**VEHICLE_TYPE**, **MANUFACTURER**, **COLOR**). 2 write locks on entities (**VEHICLE**, **PURCHASE**). 1 or more write locks on HAS_COLOR relationship. Total: 3 read locks and 3-5 write locks depending on number of **COLORs**.

Enabling Conditions: Staff subtype must be **ACQUISITION_SPECIALIST** or **OWNER**. **CUSTOMER** must be identified via **Lookup/Add Customer** task before **VEHICLE** entry. VIN must be unique. All conditions apply uniformly across the entire operation.

Frequency: Low to medium. Vehicles are bought regularly, but much less frequently than for example a search occurs. Frequency is uniform throughout the task.

Consistency (ACID): Critical. **VEHICLE** record, **PURCHASE** record, **COLOR** relationship instances, **CUSTOMER** link, and **ACQUISITION_SPECIALIST** link must get inserted atomically.

Subtasks: Mother task not needed. No decomposition required. Multiple entities and relationships are involved, but they represent a single business event with consistent enabling conditions and frequency.



Abstract Code

- ❖ User navigated to **Add Vehicle** form associated with the **Add Vehicle** task by clicking on the **Add Vehicle** button on the **Vehicle Search** form and completing the **Lookup/Add Customer** task. The **CUSTOMER** subtype and key (SSN or Tax_id), and context (Add Vehicle) get passed in from the **Lookup/Add Customer** task.

- ❖ Run the **Add Vehicle** task as follows:
 - Display **Add Vehicle** form with confirmed seller name, input fields for VEHICLE details, and Purchase_date.
 - Display dropdown fields populated from lookup tables:
 - Type showing all VEHICLE_TYPE options.
 - Manufacturer dropdown showing all MANUFACTURER options.
 - Condition with following options: Excellent, Very Good, Good, Fair, Rough.
 - Fuel Type with following options: Gas, Diesel, Natural Gas, Hybrid, Plugin Hybrid, Battery, Fuel Cell.
 - Drivetrain with options: 4WD, AWD, FWD, RWD.
 - Color with all COLOR options, with **Add Color** button for multiple colors.
 - User selects values from all of the above dropdown fields.
 - User fills in VIN, Model_name, Model_year, Horsepower, Notes (optional), Purchase_price, and Purchase_date.
 - On click **Submit** button:
 - Verify all required inputs have been provided, and validate all input fields.
 - Verify VIN is new. If VIN already exists, display error and remain on **Add Vehicle** form.
 - If valid:
 - Insert new VEHICLE record with values entered by user.
 - Link VEHICLE to MANUFACTURER via MADE_BY.
 - Link VEHICLE to VEHICLE_TYPE via IS_OF_TYPE.
 - For each COLOR added by user, link VEHICLE to COLOR via HAS_COLOR.
 - Insert new PURCHASE record with Purchase_price and Purchase_date.
 - Link PURCHASE to CUSTOMER via FROM_SELLER.
 - Link PURCHASE to user (ACQUISITION_SPECIALIST or OWNER) via HANDLED_BY.
 - Go to **Vehicle Details** form for the new VEHICLE, passing VIN.
 - On click **Cancel** button: Return to **Vehicle Search** form.

3.8 Lookup/Add Vendor

Task Decomposition

Lock Types:

1. Lookup vendor requires read-only locks on VENDOR.
2. Add vendor requires write lock on VENDOR and PARTS_ORDER.

Number of Locks: lookup vendor requires 1 read-only lock and add vendor requires 2 write locks.

Enabling Conditions: the acquisition specialist (or the owner) is successfully logged in and is using the **Vehicle Details Form** (detail page), where it is possible to launch the **Add Parts Order Form**. This form allows lookup/add vendor.

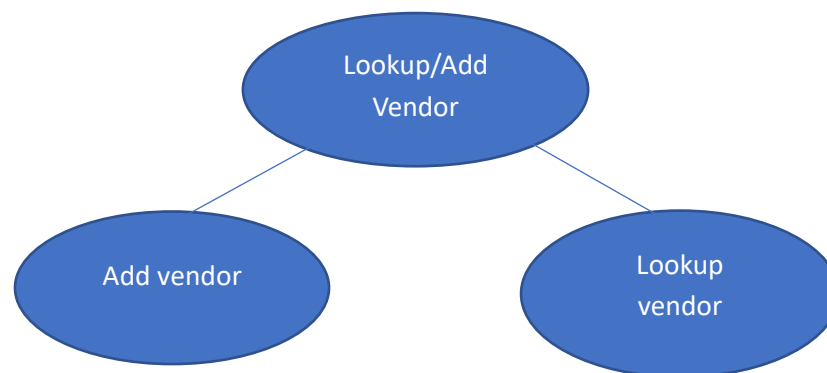
Frequency: medium-high for the read only parts, as acquisition specialists will take a significant amount of time requesting new parts, and low for the addition of vendors, which are limited.

Consistency (ACID): not critical for read-only and critical for writing actions. If we add a vendor but fail to update [PARTS_ORDER](#), we might end up corrupting the database with data relationships missing.

Subtasks: Looking at the difference in frequency of usage, as well as the lock types and the entities accessed, it seems obvious to split into two subtasks:

1. Lookup vendor: separate task because of its high frequency and only reading locks requirement.
2. Add vendor: separate task because of low frequency and write locks requirement on both entities to ensure the link is established.

We will need a mother task to handle these two subtasks.



Abstract Code

- ❖ The acquisition specialists, or the owner, successfully authenticate using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Vehicle Details Form** accessed through **Vehicle Search/Result Form**. In the **Vehicle Details Form**, the **Add Parts Order** button will launch the **Add Parts Order Form**. The **Add Parts Order Form** will enable the addition/search of Vendors using **Add/Search Vendor** button. Once the vendor is selected/added, it will be linked to the part order.
- ❖ Run the **Lookup/Add Vendor** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - The user can search for the vendor's name using **Add Parts Order** button (all [VENDOR](#).Vendor_name should be unique). The vendor's name introduced by the user is then compared to all [VENDOR](#).Vendor_name. If the search is positive, all attributes (Vendor_name, Street, Phone, Postal_code, City, and State) for that vendor [VENDOR](#) will be shown. This user could now link this vendor to current [PARTS_ORDER](#) (Parts_Order_Number + Order_ordinal) via the FULFILLS relationship.
 - In the case that the vendor does not exist in the database yet, the user can add it by filling in all required attributes for the new [VENDOR](#): Vendor_name, Street, Phone, Postal_code, City, and State. A consistency check will be performed prior to the addition of new records to [VENDOR](#) by cross checking that the introduced vendor name is not yet an existing [VENDOR](#).Vendor_name. When the vendor is added, it is automatically linked to current [PARTS_ORDER](#) (Parts_Order_Number + Order_ordinal).

- ❖ Once the user finishes searching/adding the vendor, it will be possible to go back to the **Add Parts Order Form** using the **Back to Add Parts** button. Once the user finishes adding parts, they can return to the **Vehicle Details Form** by using the **Back to Vehicle Details** button. If required, the user can go back to **Vehicle Search/Result Form** by clicking on **Back to Search** button.

3.9 Add Part to Order

Task Decomposition

Lock Types: requires write locks on **PARTS_ORDER**, and **PARTS_ORDER_LINE** and read-only lock on **VEHICLE**.

Number of Locks: add parts order requires 2 write locks and to check VIN validity we need 1 read-only lock.

Enabling Conditions: the acquisition specialist (or the owner) is successfully logged in and is using the **Vehicle Details Form** (detail page), where it is possible to launch the **Add Parts Order Form**.

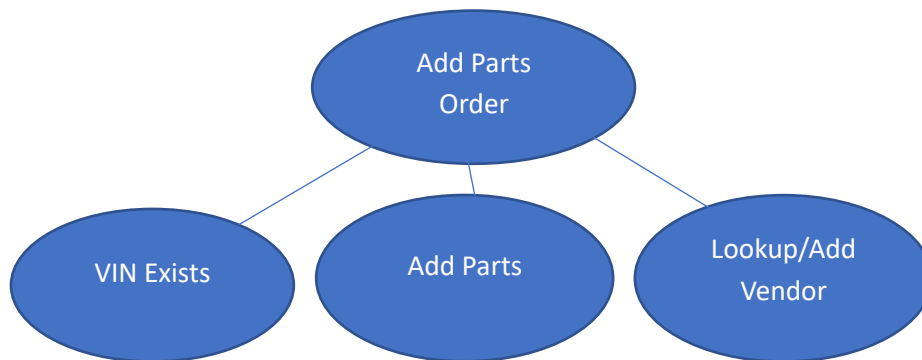
Frequency: adding parts will be one of the main tasks of the acquisition specialists, therefore the frequency of this task is considered high.

Consistency (ACID): not critical for read-only and critical for writing actions. Failing to link **PARTS_ORDER_LINE** to **PARTS_ORDER** and **VEHICLE** might corrupt the database with data relationships missing. Additionally, having duplicate parts or inconsistencies in the inventory could lead to financial and operational issues.

Subtasks: Looking at the lock types, it seems obvious to split into two subtasks:

1. VIN Exists: separate task because read-only locks are required, as well as the different entities that need to be touched (**VEHICLE**).
2. Add parts: separate task because it touches **PARTS_ORDER**, **PARTS_ORDER_LINE** and uses write locks for that.
3. Lookup/Add Vendor: is treated as a separate task and described in its own section.

We will need a mother task to handle these two subtasks.



Abstract Code

- ❖ The acquisition specialists, or the owner, successfully authenticate using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Vehicle Details Form** accessed through **Vehicle Search/Result Form**. In the **Vehicle Details Form**, the ***Add Parts Order*** button will launch the **Add Parts Order Form**.
- ❖ As explained in the previous section, the **Add Parts Order Form** will enable the addition/search of Vendors using ***Add/Search Vendor*** button.
- ❖ Run the ***Add Parts Order*** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - When the user wants to add new parts to a vehicle, a new **PARTS_ORDER** will be generated, and as many **PARTS_ORDER_LINE** as the user would like to add to the order via EXISTS_IN relationship. Additionally, **PARTS_ORDER** will be associated with **VEHICLE** via the FOR_VEHICLE Relationship.
 - The user will fill the information about the parts required for a particular order. **PARTS_ORDER.Parts_order_number** will be filled in with the VIN of the vehicle and **PARTS_ORDER.Order_ordinal** will be automatically added to the part order. **PARTS_ORDER.Order_ordinal** is initialized to one for a new VIN and is incremented by one unit (+1) every time an order part is completed.
 - As part of the process, the VIN introduced by the user will be cross checked with existing **VEHICLE.VIN** values. It will only be possible to successfully create a part order if the **VEHICLE.VIN** introduced already exists.
 - If the **VEHICLE.VIN** exists, it will be linked to current **PARTS_ORDER** (**Parts_order_number** + **Order_ordinal**) via FOR_VEHICLE relationship.
 - The user will need to keep adding information to the order part to get it completed: **PARTS_ORDER_LINE.Vendor_Part_Number**, **PARTS_ORDER_LINE.Description**, **PARTS_ORDER_LINE.Quantity**, and **PARTS_ORDER_LINE.Unit_Price**.
 - The **PARTS_ORDER_LINE.Status** will be automatically initialized to “ordered” for all newly created parts.
 - The subtask **Lookup/Add Vendor** is treated as a separate task and described in its own section. It will be used to link the order part to a **VENDOR** via the relationship FULFILLS.
- ❖ Once the user finishes adding parts, it will be possible to return to the **Vehicle Details Form** by using the ***Back to Vehicle Details*** button. If required, the user can go back to **Vehicle Search/Result Form** by clicking on ***Back to Search*** button.

3.10 Generate Seller History Report

Task Decomposition

Lock Types: Read-only on **VEHICLE**, **PURCHASE**, **CUSTOMER**, **PARTS_ORDER**, and **PARTS_ORDER_LINE**. All locks are reads to retrieve seller information and calculate summary statistics.

Number of Locks: Five read locks. **CUSTOMER** to identify sellers and retrieve names. **PURCHASE** to count vehicles per seller and calculate average purchase prices. **VEHICLE** to link purchases with parts history.

[PARTS_ORDER](#) to find parts orders per vehicle. [PARTS_ORDER_LINE](#) to calculate parts quantities and costs. All locks are on entities, no M:N relationships requiring separate locks.

Enabling Conditions: User staff subtype must be [MANAGER](#) or [OWNER](#). The report can be run at any time regardless of whether there are active sales or parts pending.

Frequency: Low. This is a management tool run to review sellers and identify problematic sellers (those that cost too much in parts). It is not part of daily operations like viewing vehicles, so it runs far less frequently than those tasks.

Consistency (ACID): Not critical. This is a historical report showing longitudinal patterns. If a new purchase or parts order is recorded while the report is being generated, it is acceptable for those new records not to be in the current report.

Subtasks: No subtasks needed.



Abstract Code

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Seller History Report** form associated with the **Generate Seller History Report** task by first clicking on the **Reports** button on the **Vehicle Search/Results** form, selecting the Seller History option on the **Reports Selection** form, and clicking the **View Report** button.
- ❖ Run the **Generate Seller History Report** task as follows:
 - Find all [CUSTOMER](#) records that have sold at least one [VEHICLE](#) via FROM_SELLER.
 - For each [CUSTOMER](#):
 - If the [CUSTOMER](#) is an [INDIVIDUAL_CUSTOMER](#), display First_name and Last_name as a single Seller_name column.
 - If the [CUSTOMER](#) is a [BUSINESS_CUSTOMER](#), display Business_name as the Seller_name column.
 - Count the number of [VEHICLES](#) sold by this [CUSTOMER](#) and display as Total_vehicles.
 - Calculate the average Purchase_price across all [PURCHASE](#) records for this Customer's [VEHICLES](#) and display as Avg_purchase_price.
 - Find all [PARTS_ORDER](#) records associated with this Customer's [VEHICLES](#) via FOR_VEHICLE.
 - For each [PARTS_ORDER](#), find all [PARTS_ORDER_LINE](#) records via CONTAINS.
 - Calculate the average total quantity of parts (sum of quantity across all [PARTS_ORDER_LINE](#) records, divided by Total_vehicles) and display as Avg_parts_quantity.

- Calculate the average total cost of parts (sum of Unit_price × Quantity across all [PARTS_ORDER_LINE](#) records, divided by Total_vehicles) and display as Avg_parts_cost.
- If Avg_parts_quantity >= 5 or Avg_parts_cost >= 500, highlight the row in red.
- Sort the results by Total_vehicles descending, then by Avg_purchase_price ascending.
- Display all rows in the report table.
- ❖ On click **Back to Reports** button: Return to the [Reports Selection](#) form.

3.11 Generate Avg Time in Inventory Report

Task Decomposition

Lock Types: Read locks on VEHICLE, VEHICLE_TYPE, PURCHASE, and SALE.

Number of Locks: 4 read locks, all on entities, no M:N relationships involved.

Enabling Conditions: User staff subtype must be Manager or Owner. The report can be run at any time regardless of whether there are active sales or parts pending.

Frequency: Low. This is a management tool run to review how long the different types of VEHICLE typically remain in inventory. It won't be changing much over time, so will not be accessed frequently by management.

Consistency (ACID): Not critical. This is a historical report showing longitudinal patterns. If a new sale is recorded while the report is being generated, it is acceptable for those new records not to be in the current report.

Subtasks: No subtasks needed.



Abstract Code

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the [Vehicle Search/Result Form](#).
- ❖ User navigated to the [Average Time in Inventory Report](#) form associated with the **Generate Average Time in Inventory Report** task by selecting the Average Time in Inventory option on the [Reports Selection](#) form and clicking the **View Report** button.
- ❖ Run the **Generate Average Time in Inventory Report** task as follows:
- ❖ Find all [VEHICLE_TYPE](#) records and display each Type_name in alphabetical order as a table row.
- ❖ For each [VEHICLE_TYPE](#):
 - Find all [VEHICLE](#) records of this type via IS_OF_TYPE that have a [SALE](#) record via SOLD_VEHICLE.

- For each sold vehicle, find the **PURCHASE** record via PURCHASED_VEHICLE to get Purchase_date.
- Find the **SALE** record via SOLD_VEHICLE to get Sale_date.
- Calculate days in inventory as $Sale_date - Purchase_date + 1$.
- Calculate the average days in inventory across all sold **VEHICLE**s of this type.
- If no vehicles of this type have been sold, display "N/A" for average days.
- Display the average days in inventory for this VEHICLE_TYPE in alphabetical order.
- On click **Back to Reports** button: Return to the **Reports Selection** form.

3.12 Generate Price Per Condition Report

Task Decomposition

Lock Types: Requires read locks on **VEHICLE**, **TYPE**, and **PURCHASE**

Number of Locks: 3 read locks.

Enabling Conditions: User staff subtype must be Manager or Owner. The report can be run at any time regardless of whether there are active sales or parts pending.

Frequency: Low. This is a management tool run to review trends in the price of vehicles based on their condition, grouped by vehicle type. It won't be changing much over time, so will not be accessed frequently by management.

Not critical. This is a historical report showing longitudinal patterns. If a new sale is recorded while the report is being generated, it is acceptable for those new records not to be in the current report.

Subtasks: No subtasks needed.



Abstract Code

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Generate Price per Condition Report** form associated with the **Generate Price per Condition Report** task by selecting Monthly Sales/Drilldown on the **Reports Selection** form and clicking the **View Report** button. **Reports Selection** can be accessed from **Vehicle Search/Result Form**.
- ❖ Run the **Generate Price per Condition Report** task as follows:
 - Display a table with 5 rows (one for each Condition) and 9 columns (one for each vehicle type)

- Populate the first column with the 5 possible values of the Condition attribute, in descending order (Excellent first).
- The column names should consist of 'Condition' followed by the 8 vehicle types in alphabetical order.
- For each cell in the matrix (each Condition-Type pair):
 - Find all VEHICLE records whose Condition matches the row and whose TYPE matches the column name via IS_OF_TYPE.
 - For each matching VEHICLE, find its PURCHASE record via PURCHASED_VEHICLE and retrieve its Purchase_price attribute.
 - Calculate the average of all Purchase_price attributes for that combination of condition and vehicle type.
 - Display the average Purchase_price in the cell.
 - If no matching records found, display \$0 in the cell.
- ❖ On click **Back to Reports** button: Return to the **Reports Selection** form.

3.13 Generate Parts Statistics Report

Task Decomposition

Lock Types: requires read-only locks on [PARTS_ORDER](#), [PARTS_ORDER_LINE](#), and [VENDOR](#) to build the report.

Number of Locks: 3 read-only locks.

Enabling Conditions: the operating manager, or the owner, is successfully logged in and is accessing the **Parts Statistics Report**.

Frequency: low, as it is expected that the owner or the operating managers will run this kind of report once or twice a day at most.

Consistency (ACID): not critical, as the report just compiles information and temporary inconsistencies would not have any financial or operational implications.

Subtasks: no mother task/subtask decomposition is needed.



Abstract Code

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.

- ❖ User navigated to the **Parts Statistics Report** form associated with the **Parts Statistics Report** task by selecting Parts Statistics on the **Reports Selection** form and clicking the **View Report** button. **Reports Selection** can be accessed from **Vehicle Search** form.
- ❖ Run the **Generate Parts Statistics Report** task using the HTTP Session/Cookie associated to the user as follows:
 - Find all **VENDOR** records and retrieve the associated **PARTS_ORDER** records via the FULFILLS relationship and **PARTS_ORDER_LINE** records via the EXISTS_IN relationship using **VENDOR**.Vendor_name identifying property as classifier.
 - For each **VENDOR**.Vendor_name:
 - Find the total quantity of parts coming from a particular **VENDOR**.Vendor_name, using **PARTS_ORDER**.Parts_order_number and **PARTS_ORDER**.Order_ordinal to identify the parts linked to a particular **VENDOR**.Vendor_name and adding **PARTS_ORDER_LINE**.Quantity for all those items.
 - Find the total dollar amount spent on parts coming from a particular **VENDOR**.Vendor_name, using **PARTS_ORDER**.Parts_order_number and **PARTS_ORDER**.Order_ordinal to identify the parts linked to a particular **VENDOR**.Vendor_name and adding **PARTS_ORDER_LINE**.Unit_price multiplied by **PARTS_ORDER_LINE**.Quantity for all those items.
 - Display **VENDOR**.Vendor_name ordered in alphabetical order together with the number of parts coming from that vendor and the total dollar amount spent on parts.
- ❖ Once the user finishes reading the report, it will be possible to go back to **Reports Selection** using the **Back to Reports** button.
- ❖ From **Reports Selection** the user is allowed to go back to Vehicle Search form by clicking on **Back to Vehicle Search** button.

3.14 Generate Monthly Sales/Drilldown Report

Task Decomposition

Lock Types: requires read-only locks on **VEHICLE**, **SALE**, **PURCHASE**, **PARTS_ORDER**, and **PARTS_ORDER_LINE** to build the monthly sales report, and read-only locks on **SALES_AGENT**, **SALE**, and **STAFF** to build the drilldown reports for month and year.

Number of Locks: 4 read-only locks to build the monthly sales report and 3 read-only to build the drilldown reports.

Enabling Conditions: the operating manager, or the owner, is successfully logged in and is accessing the **Monthly Sales Report**.

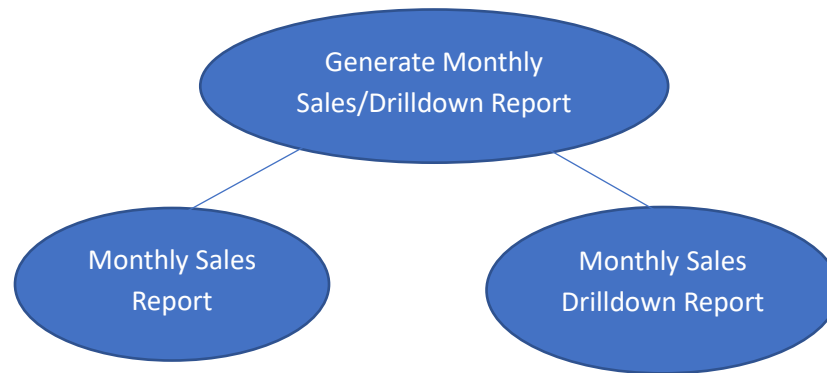
Frequency: medium-high, as this is the most frequently used report.

Consistency (ACID): not critical, as the report just compiles information and temporary inconsistencies would not have any financial or operational implications.

Subtasks: as we are using different entities to build the reports, we will separate into two subtasks:

1. Monthly sales report: separate task because different entities ([VEHICLE](#), [SALE](#), [PURCHASE](#), [PARTS_ORDER](#), and [PARTS_ORDER_LINE](#)) are required to build this report, as compared with the drilldown report
2. Monthly sales drilldown report: separate task because it touches different entities ([SALES_AGENT](#), [SALE](#), and [STAFF](#)).

We will need a mother task to handle these two subtasks.



Abstract Code

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Monthly Sales/Drilldown Report** form associated with the **Generate Monthly Sales/Drilldown Report** task by selecting Monthly Sales/Drilldown on the **Reports Selection** form and clicking the ***View Report*** button. **Reports Selection** can be accessed from **Vehicle Search/Result Form**.
- ❖ Run the **Generate Monthly Sales/Drilldown Report** task using the HTTP Session/Cookie associated to the user as follows:
 - Find all [SALE](#) records and retrieve the associated [VEHICLE](#) via the SOLD_VEHICLE relationship and classify them by year and month/year based on [SALE](#).Sale_date attribute. Find all [PARTS_ORDER](#) associated with those [VEHICLE](#) via the FOR_VEHICLE relationship and retrieve [PARTS_ORDER_LINE](#) records via the EXISTS_IN relationship to calculate vehicle costs:
 - For each period (year and month):
 - Find the total number of vehicles sold in each period by counting [SALE](#).Sale_date.
 - Find the *gross sales income* by adding [VEHICLE](#).Sale_price for all [VEHICLE](#).VIN sold in each period.
 - Find the *vehicle costs* by adding [VEHICLE](#).Purchase_price and [PARTS_ORDER_LINE](#).Unit_price multiplied by [PARTS_ORDER_LINE](#).Quantity for [PARTS_ORDER](#).Parts_order_number associated with all [VEHICLE](#).VIN sold in each period.
 - Calculate the *total net income* by subtracting the *vehicle costs* from the *gross sales income* for the specified period.

- Display number of sales, the *gross sales income* and the *total net income* for each period.
- Find all [SALE](#) records and retrieve the associated [SALES_AGENT](#) via the EXECUTES relationship and the [STAFF](#) details via the overlapping specialization.
 - In the case that the user would like to obtain a drilldown report for a given period, the **Drilldown** button is available to perform following calculations:
 - Obtain the [SALES_AGENT.STAFF.User_name](#) for all [VEHICLE.VIN](#) sold in each period and count the number of sales.
 - Calculate *gross sales income* (adding [VEHICLE.Sale_price](#)) for all [VEHICLE.VIN](#) sold in each period for a particular [SALES_AGENT.STAFF.User_name](#).
 - The drilldown report will display [SALES_AGENT.STAFF.First_name](#) and [SALES_AGENT.STAFF.Last_name](#), the number of vehicles sold, and the *gross sales income* for the period selected. The sales agents list will be ordered by the number of vehicles sold (descending) followed by *gross sales income* (descending).
 - Once the user finishes reading the drilldown report, it will be possible to go back to the **Monthly Sales/Drilldown Report** using the **Back to Reports** button.
- ❖ Once the user finishes reading the **Monthly Sales/Drilldown Report**, it will be possible to go back to **Reports Selection** using the **Back to Reports** button.
- ❖ From **Reports Selection** the user is allowed to go back to **Vehicle Search/Result Form** form by clicking on **Back to Vehicle Search** button.